

Trabajo Práctico

El representante comercial de un importante restaurante ubicado en la zona sur del Gran Buenos Aires ha solicitado sus servicios para desarrollar un **sistema de menú digital**. Debido a los cambios que debieron implementar los restaurantes en la actualidad, dejaron de compartir la carta física con los clientes, por lo que necesitan una aplicación web que permita visualizar toda la información que contenía la carta tradicional y, además, gestionar las **comandas** de los clientes.

El sistema debe permitir que los clientes puedan consultar el menú de manera clara y organizada, visualizando categorías, platos disponibles y sus precios. Asimismo, debe contemplar la creación de comandas en las que se puedan **agregar platos, definir cantidades y añadir notas** (por ejemplo, indicar la cocción deseada de la carne o excluir algún ingrediente).

Para llevar un control del servicio, el sistema debe permitir **identificar la comanda** según corresponda:

- Si el pedido es para consumir en el salón, se debe ingresar el número de **mesa**.
- Si el pedido es para llevar, se debe ingresar el **nombre del comensal**.
- Si el pedido es delivery, se debe ingresar una **dirección**

El personal del restaurante debe poder acceder a las comandas generadas, visualizar su estado e **ir modificándolo** a medida que avanza el servicio (por ejemplo: *Pendiente, En preparación, Listo, Entregado*), llevando así un control de las órdenes en curso.

La entrega de esta app se realiza en tres etapas:

1. Entrega de prototipo con lógica de base de datos (Parte 1)
2. Entrega de Backend con la lógica de negocios (Parte 2)
3. Entrega de Página web (Parte 3)

Parte 1

Criterios de aceptación:

1. Debe permitir crear platos
2. Debe enlistar la información de los platos y permitir filtrar por nombre y/o categoría y ordenar por precio.
3. Debe permitir actualizar la información de un plato

Consigna:

Realizar una aplicación con la arquitectura API Rest, que exponga los servicios necesarios para cumplir con los criterios de aceptación. La misma debe utilizar base de datos relacional.

- La aplicación se debe ajustar al estándar de REST. Tanto los métodos de petición como los de respuesta.
- Los endpoints deben respetar la siguiente definición de [OpenApi](#). Pueden ver la definición utilizando la herramienta de [Swagger UI](#)
- Se debe utilizar un ORM.
- El modelo de datos debe ser como el que visualiza la figura 1
- Se debe utilizar Code-First para crear el modelo de base de datos.
- Se debe precargar en la base de datos los datos solicitados
- Las url, parámetros, body, status y response definidos en la definición de OpenApi deben respetarse.
- El filtro de orden sólo puede admitir los valores "ASC" y "DESC"
- No puede existir platos con el mismo nombre
- Los filtros son siempre opcionales

La aplicación puede ser desarrollado en:

- Java -> Spring Boot - Hibernate.
- C# -> Entity Framework Core
- Python -> SQLAlchemy
- Go -> GORM
- Node.js -> Sequelize

Modelo de base de datos:

El modelo a implementar será el siguiente.

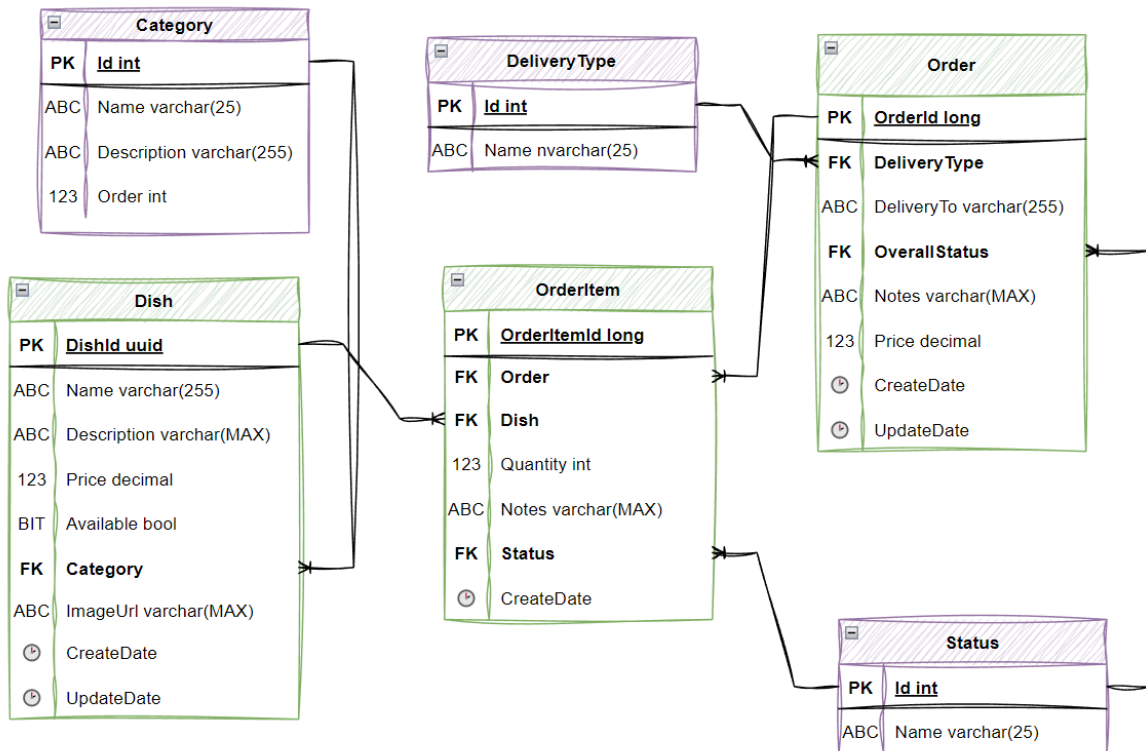


Figura 1: modelo de base de datos

Precarga de datos:

DeliveryType	
Id	Name
1	Delivery
2	Take away
3	Dine in

Status	
Id	Name
1	Pending
2	In progress
3	Ready
4	Delivery
5	Closed

Category			
Id	Name	Description	Order
1	Entradas	Pequeñas porciones para abrir el apetito antes del plato principal.	1
2	Ensaladas	Opciones frescas y livianas, ideales como	2

		acompañamiento o plato principal.	
3	Minutas	Platos rápidos y clásicos de bodegón: milanesas, tortillas, revueltos.	3
4	Pastas	Variedad de pastas caseras y salsas tradicionales.	5
5	Parrilla	Cortes de carne asados a la parrilla, servidos con guarniciones.	4
6	Pizzas	Pizzas artesanales con masa casera y variedad de ingredientes.	7
7	Sandwiches	Sandwiches y lomos completos preparados al momento.	6
8	Bebidas	Gaseosas, jugos, aguas y opciones sin alcohol.	8
9	Cerveza Artesanal	Cervezas de producción artesanal, rubias, rojas y negras.	9
10	Postres	Clásicos dulces caseros para cerrar la comida.	10

Entrega:

El código debe ser subido al campus virtual o enviado por mail al docente en formato ZIP o RAR con el nombre: "TP1-Apellido_Nombre"

Pautas de evaluación:

1. **Criterios de aceptación [4 pts]**
2. **Calidad del código [4 pts]**
 - a. **Clean Code**
 - b. **Clean Architecture**
 - c. **Solid**
 - d. **Funcionalidad y usabilidad**
3. **Base de datos [2 pts]**

RECURSOS:

1. Tutorial SQLAlchemy: <https://auth0.com/blog/sqlalchemy-orm-tutorial-for-python-developers/>
2. Tutorial Java Spring Boot : <https://www.arquitecturajava.com/spring-boot-jpa-y-su-configuracion/>
3. Tutorial EF core C#: <https://www.entityframeworktutorial.net/efcore/entity-framework-core-console-application.aspx>
4. Tutorial GORM: <https://gorm.io/docs/index.html>
5. Tutorial Sequelize: <https://sequelize.org/docs/v6/getting-started/>

6. Tutorial API Net Core:
<https://docs.microsoft.com/en-us/aspnet/core/tutorials/first-web-api?view=aspnetcore-3.1&tabs=visual-studio>
7. Tutorial API Spring Boot:
<https://www.nigmacode.com/java/Crear-API-REST-con-Spring>
8. Tutorial API Flask: <https://flask-restful.readthedocs.io/en/latest/quickstart.html>
9. Tutorial Go:
<https://blog.devgenius.io/simple-rest-service-in-golang-with-openapi-spec-and-orm-a447b1086e21>
10. Tutorial Node JS:
<https://www.freecodecamp.org/news/how-to-build-explicit-apis-with-openapi/>

Parte 2

Los criterios de aceptación son:

1. Se debe poder listar la información de “Category”, “DeliveryTypes” y “Status”
2. Debe permitir registrar las órdenes (el pedido del cliente)
3. Debe enlistar las órdenes con el detalle de los platos según la fecha que se le ingrese y estado.
4. Debe permitir actualizar el estado de un ítem de la orden
5. Debe permitir agregar platos a la orden.
6. Debe permitir eliminar el plato.
7. Se debe poder acceder a la información de un plato.
8. Se debe poder consultar la información de una comanda.

Consigna:

Realizar una aplicación con la arquitectura API Rest, que exponga los servicios necesarios para cumplir con los criterios de aceptación.

Se deberá reescribir la aplicación de consola realizada en el TP 1 y adaptarla a los nuevos requerimientos.

- La aplicación se debe ajustar al estándar de REST. Tanto los métodos de petición como los de respuesta.
- Los endpoints deben respetar la definición de [OpenApi](#). Pueden ver la definición utilizando la herramienta de [Swagger UI](#)

Aclaración:

- Las url, parámetros, body, status y response definidos en la definición de OpenApi deben respetarse.
- Los endpoints con método HTTP [Options] definido en el archivo, deben ser reemplazado por el estudiante según el estándar REST.
- El plato no puede ser eliminado si existe una orden que dependa de esta.
- Los filtros son siempre opcionales
- Al crear una orden, se debe calcular el precio y comenzar en estado pendiente, tanto el ítem como la orden.
- La transición de estados de la orden depende del estado de sus ítem, solo si todos los ítems de la orden transicionan a un mismo estado, la orden debe transicionar de estado.

Entrega:

Esta práctica debe ser entregada antes del cierre del sprint actual.

El código debe ser subido al campus virtual o enviado por mail al docente en formato ZIP o RAR con el nombre: "TP2-Apellido_Nombre"

Pautas de evaluación:

1. **Criterios de aceptación [4 pts]**
2. **Calidad del código [4 pts]**
 - a. **Clean Code**
 - b. **Clean Architecture**
 - c. **Solid**
 - d. **Funcionalidad y usabilidad**
3. **Estándares API Restful [2 pts]**

RECURSOS:

1. Tutorial API Net Core:
<https://docs.microsoft.com/en-us/aspnet/core/tutorials/first-web-api?view=aspnetcore-3.1&tabs=visual-studio>
2. Tutorial API Spring Boot:
<https://www.nigmacode.com/java/Crear-API-REST-con-Spring>
3. Tutorial API Flask: <https://flask-restful.readthedocs.io/en/latest/quickstart.html>
4. Tutorial Go:
<https://blog.devgenius.io/simple-rest-service-in-golang-with-openapi-spec-and-orm-a447b1086e21>
5. Tutorial Node JS:
<https://www.freecodecamp.org/news/how-to-build-explicit-apis-with-openapi/>

Parte 3

Los criterios de aceptación son:

1. **Visualización del menú:** El usuario podrá recorrer todas las categorías del menú y acceder al detalle de cada plato, donde se mostrará información completa como nombre, descripción, precio, disponibilidad e imágenes
2. **Creación de comandas:** El usuario podrá armar su comanda seleccionando los platos deseados, indicando cantidades y agregando notas personalizadas (por ejemplo: retirar un ingrediente o definir el punto de cocción). Además, deberá elegir la forma de entrega (*en mesa o para llevar*).
3. **Búsqueda y filtrado:** La aplicación permitirá realizar búsquedas y aplicar filtros sobre los platos facilitando la navegación dentro del menú.
4. **Gestión de comandas:** El usuario podrá consultar sus comandas activas, agregar nuevos platos a una comanda ya iniciada y revisar el detalle completo de su pedido.
5. **Panel de órdenes:** La aplicación dispondrá de un panel centralizado donde el personal del restaurante podrá visualizar todas las órdenes en curso y modificar su estado (Pendiente, En preparación, Listo, Entregado), asegurando el control y seguimiento en tiempo real.

Consigna:

Realizar una aplicación de frontend utilizando HTML, CSS y JS que cumpla con los requerimientos del cliente.

- Utilizar la API Rest realizada en el trabajo práctico 2 para suministrar los datos a la aplicación de Frontend.
- La página web debe contar con un buen diseño de UI. “El buen diseño de interfaz de usuario admite usabilidad con diseños visuales claros y consistentes y estructuración de contenido”(link). Se puede utilizar bootstrap o otro framework para el diseño de la interfaz.
- La página web debe ser intuitiva para el usuario y tener una buena UX

Pautas de evaluación:

1. **Criterios de aceptación [2 pts]**
2. **Calidad del código [2 pts]**
3. **Diseño de UI [4 pts]**
4. **UX [2 pts]**

Entrega:

Esta práctica debe ser entregada antes del cierre del sprint actual.

El código debe ser subido al campus virtual o enviado por mail al docente en formato ZIP o RAR con el nombre: “TP3-Apellido_Nombre”

RECURSOS:

1. Tutorial HTML : <https://www.w3schools.com/html/default.asp>
2. Tutorial CSS : <https://www.w3schools.com/css/default.asp>
3. Tutorial JS: <https://www.w3schools.com/js/default.asp>
4. Framework de UI:
 - a. Bootstrap: <https://getbootstrap.com/>
 - b. Foundation: <https://get.foundation/sites.html>
 - c. Pure: <https://purecss.io/>
 - d. Skeleton: <http://getskeleton.com/>